

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. OVERVIEW OF CONTRIBUTIONS AND ARTIFACTS

A. Paper’s Main Contributions

- C_1 OCEAN is a software emulation framework for CXL 3.0 that implements GFAM, MH-SLD for as many as 16 hosts, Dynamic Capacity Devices, fabric management, active latency injection, token-bucket bandwidth control, and MESI-based software cache coherence with distributed snoop filters.
- C_2 OCEAN provides a scalable multi-host fabric architecture with configurable CXL switches, topology discovery and address allocation, dynamic capacity management, and SHM, TCP, and RDMA transports.
- C_3 OCEAN exposes emulated CXL memory through the Linux CXL, NUMA, and DAX interfaces, allowing existing applications and system software to run without source changes.
- C_4 OCEAN is evaluated against real CXL Type-3 hardware using LMBench, STREAM, and OSU microbenchmarks. It is then used to study hardware cache-coherence coverage with Tigon/TPC-C and YCSB, twelve memory-placement policies with GROMACS, and calibration of the inter-host LogP timing model.

B. Computational Artifacts

- A_1 The top-level artifact harness, OCEAN source and modified QEMU, guest kernel and disk image, LMBench, MPI STREAM, OSU Micro-Benchmarks, the MPI CXL shim, and the experiment and plotting scripts for Figs. 3–5. The release archive is identified by DOI <https://doi.org/10.5281/zenodo.21844690>.
- A_2 SimCXL/gem5 source, its downloaded kernel and disk image, the ASIC and FPGA timing profiles, and the SimCXL portion of the Figure 3 runner.
- A_3 The OCEAN application and calibration workflow for Figs. 6–9. It includes configuration, collection, validation, provenance, and plotting code. Tigon, GROMACS/PEPSIN, and the physical-CXL platform are external inputs identified below.

Artifact	Contributions	Reproducible paper elements
A_1	C_1 – C_4	OCEAN deployment and Figs. 3–5
A_2	C_4	SimCXL ASIC/FPGA series in Fig. 3
A_3	C_4	Figs. 6–9 and Table II

II. ARTIFACT IDENTIFICATION

A. Computational Artifact A_1

Relation To Contributions

The artifact has two evaluation interfaces. The repository-level `run-experiments.sh` drives Figs. 3–5. With no

arguments it attempts fresh execution of all three figures. Its `--replot` mode never boots a VM or runs a benchmark: it reads the supplied CSV files and regenerates the plots. Passing `--replot 3`, `--replot 4`, or `--replot 5` selects one figure.

Figs. 6–9 use the configuration-driven program `Ocean/script/reproduce_figures_6_9.py`. This second interface separates prerequisite checking, collection, validation, and plotting and writes each successful run to `Ocean/artifact/figures_6_9/<run-id>/`. The split is intentional: Figs. 3–5 have self-contained VM runners, whereas Figs. 6–9 require site-specific application commands and, for Fig. 9, a physical shared-CXL platform.

The release also retains `artifacts/figures_6_9_live/` as bring-up evidence. Despite its historical name, that directory is not a validated Figs. 6–9 result bundle: the supplied snapshot contains server and VM console logs plus provisional Figure 5 TCP/OSU diagnostics, but no complete TPC-C, YCSB, GROMACS, or LogP tables. Reviewers must use only a validated `normalized/` subdirectory from a successful run under `Ocean/artifact/figures_6_9/<run-id>/` as plotting input.

Expected Results

After deployment, every OCEAN guest should enumerate an emulated CXL Type-3 device and expose either a CXL NUMA node or a DAX device such as `/dev/dax0.0`. The evaluation checks the following outcomes:

- the Figure 3 OCEAN and SimCXL LMBench curves reproduce the expected cache-to-CXL transition;
- Figure 4 quantifies how shared-pool STREAM bandwidth changes from two to six OCEAN hosts, and Figure 5 reports two-host OSU Allgather latency from 2 B through 16 KiB;
- increasing hardware-coherence coverage substantially improves TPC-C NewOrder throughput, and software-coherent protocols degrade for write-heavy YCSB mixes;
- the twelve GROMACS placement policies produce a measurable execution time spread (up to 7% in the paper), with SHM faster than TCP; and
- calibrated LogP parameters reproduce measured point-to-point latency and contention trends substantially better than the defaults.

Replotting confirms the plotting code and the correspondence between the archived tables and figures. It does not constitute a fresh measurement. Likewise, passing unit tests or a prerequisite check for Figs. 6–9 does not substitute for successful Tigon, GROMACS, or physical-CXL execution.

Expected Reproduction Time (in Minutes)

Tool setup is the longest stage because it compiles OCEAN’s QEMU fork and SimCXL/gem5 and downloads

large guest images and kernels. Figure 3’s gem5 runs and the application experiments in Figs. 6–8 may take hours. Figure 9 also requires scheduled access to the physical-CXL testbed. Plot-only mode is the appropriate short evaluation path when those resources are unavailable.

Artifact Setup (incl. Inputs)

Hardware: Figs. 3–5 require an x86-64 Linux server with KVM, root access for bridge/TAP and NUMA configuration, enough memory for as many as six QEMU guests, and enough disk space for the downloaded images and private VM overlays. Figure 3 also needs the CPU and storage capacity to run gem5. Figures 4 and 5 require working guest-to-guest SSH and MPI connectivity; Figure 5 uses two guests, whereas the Figure 4 wrapper defaults to at most six.

The paper’s reference hardware for Fig. 9 consists of two compute nodes, each with two Intel Xeon Gold 5420+ sockets (28 cores/socket) and 1 TB DRAM, connected via PCIe 5.0 to an FPGA-based CXL memory system providing a 32 GB shared pool. Fresh Fig. 9 measurements require two hosts mapping the same CXL Type-3 DAX range. A machine without that topology may validate and plot an archived normalized result bundle but cannot reproduce the hardware measurements.

Software: The setup supports native Linux and uses Bash, Git, CMake, Make, SCons, GCC/G++, Python 3, KVM/QEMU, NumPy, Matplotlib, OpenMPI, Lmbench, STREAM, and OSU Micro-Benchmarks. Figure 3 additionally requires SimCXL/gem5. Figures 6 and 7 require Tigon, Figure 8 requires GROMACS/PEPSIN and the MPI CXL shim, and Figure 9 requires MPI plus the optional `cxl_switch_lock_bench_mpi` target. Exact package and revision information should be recorded with each fresh result bundle.

Datasets / Inputs: Lmbench, STREAM, and OSU generate synthetic inputs. In the OCEAN guest, Figure 3 uses a 1024 MB Lmbench allocation, 64-byte stride, four repetitions, and NUMA binding to the emulated CXL node. Figure 4 runs MPI STREAM Copy, Scale, Add, and Triad while varying the number of hosts up to six. Figure 5 runs OSU MPI Allgather on two VMs for message sizes 2^1 through 2^{14} bytes.

Figures 6 and 7 use Tigon’s TPC-C and YCSB inputs on two OCEAN nodes. Figure 8 uses the PEPSIN input with a baseline and twelve alternative placement policies under SHM and TCP. Figure 9 operates on a deliberately bounded, shared DAX range from two physical hosts. These application inputs and commands are supplied through the example TOML configuration; the evaluator must replace its placeholder workload paths and runners before collection.

Installation and Deployment: Start at the repository root. The relevant layout is:

```
artifacts/setup-tools.sh
artifacts/tools/OCEAN/
artifacts/tools/SimCXL/
artifacts/fig3/ ... artifacts/fig5/
Ocean/script/reproduce_figures_6_9.py
run-experiments.sh
```

The complete release archive is identified by <https://doi.org/10.5281/zenodo.21844690>. The source and small artifact files are packaged in that archive; the large OCEAN guest resources are downloaded separately. The canonical resource links are:

- `qemu.img`: https://drive.google.com/file/d/19yLZPEI5HN23noVx2m3OsYhJVbuvan1y/view?usp=drive_link;
- `bzImage`: https://drive.google.com/file/d/1hxHVrcPfoO-PRbWFhYJEala7UUVL7hJoy/view?usp=drive_link.

Install system dependencies on a dedicated machine, then perform the common setup:

```
./artifacts/setup-tools.sh --stage deps
./artifacts/setup-tools.sh
```

The first command is separated because package installation requires `sudo`. The default setup initializes the required OCEAN and SimCXL sources, builds SimCXL/gem5, downloads the SimCXL kernel and disk image, runs `tools/OCEAN/script/setup_host.sh` to build OCEAN’s QEMU fork and host dependencies, and compiles the OCEAN server. The artifact contract installs the guest resources from the links above as `artifacts/tools/OCEAN/build/qemu.img` and `artifacts/tools/OCEAN/build/bzImage`; the experiment wrappers consume those paths without manual launcher edits. Record a SHA-256 checksum for each downloaded file in the result manifest. Because setup and experiment scripts install packages, create bridge/TAP interfaces, and launch KVM guests, they should run only on a dedicated evaluation host.

Artifact Execution

After setup, a full fresh run of Figs. 3–5 is:

```
./run-experiments.sh
```

The driver attempts Figures 3, 4, and 5 in order. To regenerate plots from the supplied data without launching experiments, use:

```
./run-experiments.sh --replot
./run-experiments.sh --replot 3
./run-experiments.sh --replot 4
./run-experiments.sh --replot 5
```

Figs. 6–9 use a separate configuration. Copy the example, fill in the Tigon and GROMACS commands and the physical-CXL parameters, and run a non-mutating prerequisite check before collection:

```
cp Ocean/script/figures_6_9/config.example.toml \
  figures_6_9.toml
python3 Ocean/script/reproduce_figures_6_9.py \
  doctor --fig all --config figures_6_9.toml
python3 Ocean/script/reproduce_figures_6_9.py \
  all --fig all --config figures_6_9.toml
```

Individual figures are selected with `--fig 6`, `--fig 7`, `--fig 8`, or `--fig 9`. Existing normalized Figs. 6–9 data can be validated and plotted without running a workload by using the `validate` and `plot` subcommands described in Evaluation 3.

Artifact Analysis (incl. Outputs)

A successful Figures 3–5 run leaves raw logs under `artifacts/logs/`, normalized CSV files under each figure’s `data/` directory, and PNG plots beside those tables. A successful Figures 6–9 run contains raw command logs, normalized CSV tables, PDF and PNG plots, and a checksum-bearing `manifest.json` under one run-id. The Figs. 6–9 workflow fails closed when a sweep is incomplete, a workload exits nonzero, GROMACS reports zero work, MPI rank metadata is missing, or Fig. 9’s DAX authorization is absent.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Tool setup and plot-only smoke test.

Artifact Setup (incl. Inputs)

This evaluation checks that the artifact is complete before spending time on VMs or `gem5`. Plotting requires Python 3 and Matplotlib but does not require KVM, root privileges, Tigon, GROMACS, or physical CXL hardware. Obtain the artifact from the persistent DOI listed in the AD, install the Python requirements, then replot all supplied Figures 3–5 data:

```
python3 -m pip install -r requirements.txt
./run-experiments.sh --replot
```

Artifact Execution

Confirm that individual selection invokes only the requested plotter:

```
./run-experiments.sh --replot 3
./run-experiments.sh --replot 4
./run-experiments.sh --replot 5
```

For Figs. 6–9, run the OCEAN unit tests and the non-mutating prerequisite check. The example configuration intentionally contains placeholders, so the `doctor` command may report unavailable workloads or hardware; this is an informative result rather than measured reproduction:

```
python3 -m pytest Ocean/tests/figures_6_9
cd Ocean
python3 script/reproduce_figures_6_9.py doctor \
--fig all \
--config script/figures_6_9/config.example.toml
```

Do not pass `artifacts/figures_6_9_live/` to the plotting command. It documents attempted VM/TCP bring-up and is intentionally outside the normalized result contract.

Artifact Analysis (incl. Outputs)

Pass criteria are that every requested Figures 3–5 plot is created and each plot command exits zero. The Figures 6–9 unit tests must pass. A successful `doctor` additionally proves that the configured workload executables, hostfile, Python modules, MPI benchmark, and DAX character device are present. None of these checks alone proves a fresh performance result.

B. Computational Artifact A_2

Fresh microbenchmark execution (Figs. 3–5).

Artifact Setup (incl. Inputs)

Use a dedicated Linux/KVM server and run the setup script:

```
./artifacts/setup-tools.sh
```

The script initializes OCEAN and SimCXL and their dependencies, builds OCEAN’s QEMU fork and SimCXL/`gem5`, downloads the required kernels and disk images, and places those resources where the experiment wrappers expect them. Figure 3 requires one OCEAN VM and separate SimCXL simulations. Figure 4 uses up to six OCEAN VMs. Figure 5 uses two OCEAN VMs with guest-to-guest networking and MPI.

If the automated resource stage cannot access Google Drive, use the canonical `qemu.img` and `bzImage` download links listed in the AD. Place both files in the OCEAN build directory; then verify that they are nonempty and record their hashes:

```
test -s artifacts/tools/OCEAN/build/qemu.img
test -s artifacts/tools/OCEAN/build/bzImage
sha256sum artifacts/tools/OCEAN/build/qemu.img \
artifacts/tools/OCEAN/build/bzImage
```

Artifact Execution

Invoke all three experiments with the default command:

```
./run-experiments.sh
```

For Fig. 3, the wrapper discovers the guest’s emulated CXL NUMA node and runs the following command inside the single OCEAN VM:

```
numactl --cpunodebind=0 --membind=${CXL_NODE} \
./lat_mem_rd -t -N 4 1024 64
```

This binds the benchmark thread to CPU NUMA node 0, allocates memory from the emulated CXL node, performs four timed repetitions, uses a 1024 MB maximum working set, and advances at 64-byte stride. The same figure wrapper runs the SimCXL ASIC and FPGA profiles and parses their LMBench output before producing the comparison plot.

For Fig. 4, the wrapper caps the experiment at six hosts by default. It creates the OCEAN guests and copies this MPI STREAM workload to the VMs:

```
artifacts/tools/OCEAN/workloads/stream
```

It constructs a hostfile containing the active nodes and runs STREAM through `libmpi_cxl_shim.so`. The shim allows OCEAN to intercept the MPI memory operations. The runner accepts an explicit host list for smaller runs; an evaluator with sufficient resources may deliberately request more than six hosts.

For Fig. 5, the wrapper creates two VMs and a two-entry MPI hostfile. The OSU Allgather binary is already present in the supplied guest image, so no workload copy is needed. It preloads `libmpi_cxl_shim.so`, binds the shim to the emulated CXL DAX device, and runs Allgather for message sizes from 2^1 to 2^{14} bytes.

Artifact Analysis (incl. Outputs)

The expected machine-readable outputs are the Figure 3 OCEAN, SimCXL ASIC, and SimCXL FPGA CSV files; the Figure 4 STREAM CSV containing host count, kernel, throughput, and timing fields; and the Figure 5 CSV containing

transport, message size, and average latency. Each runner also creates a PNG plot and retains timestamped logs. A missing series, skipped host count, failed MPI command, or empty result table must be reported as an incomplete experiment, not silently replaced with archived data.

C. Computational Artifact A_3

Application experiments (Figs. 6–8).

Artifact Setup (incl. Inputs)

Create a working configuration from the supplied template. Figures 6 and 7 require a working Tigon checkout and its database image. Figure 8 requires the PEPSIN input, GROMACS, and an executable runner for every SHM/TCP and placement-policy combination. Configure two distinct OCEAN VMs with working peer networking. Before collection, run:

```
python3 Ocean/script/reproduce_figures_6_9.py \
doctor --fig all --config figures_6_9.toml
```

Artifact Execution

For Fig. 6, run TPC-C NewOrder on two nodes using 2PL and sweep HCC coverage over 0, 25, 50, 70, 80, 90, and 100 percent. Tigon must be invoked with the persistence/WAL settings required by the supplied runner, including:

```
--persist_latency=0
--wal_group_commit_time=0
--wal_group_commit_size=0
```

The collector requires a throughput record for each of the two nodes at every configured coverage value.

For Fig. 7, sweep the write ratio from 0 through 100 percent in steps of 10 for Tigon, DS2PL+, and Sundial+. For Fig. 8, run the baseline and twelve alternative placement policies for both SHM and TCP. Keep rank count, thread count, PEPSIN input, MD steps, CPU binding, and memory capacity fixed. The collector rejects fatal GROMACS output, missing completion markers, zero created threads, and nonpositive elapsed time.

Collect, validate, and plot the three figures with:

```
python3 Ocean/script/reproduce_figures_6_9.py \
all --fig 6 --config figures_6_9.toml
python3 Ocean/script/reproduce_figures_6_9.py \
all --fig 7 --config figures_6_9.toml
python3 Ocean/script/reproduce_figures_6_9.py \
all --fig 8 --config figures_6_9.toml
```

Artifact Analysis (incl. Outputs)

Each successful command creates raw logs, a normalized CSV table, plots, and a manifest. Fig. 6 should show increasing TPC-C throughput with increasing HCC coverage. Fig. 7 should expose the different sensitivity of the three protocols to write-heavy mixes. Fig. 8 should contain all thirteen policy labels for both backends and a nonzero execution-time measurement for every point. Cross-machine numerical differences are acceptable, but incomplete sweeps are not. The provisional files under artifacts/figures_6_9_live/ do not satisfy these pass criteria.

D. Computational Artifact A_4

LogP calibration (Fig. 9 and Table II).

Artifact Setup (incl. Inputs)

This evaluation requires two physical hosts mapping the same CXL Type-3 DAX range through a CXL switch, MPI, cache-flush instructions, and the optional MPI lock benchmark. Record the host, CPU, CXL device, switch, firmware, DAX range, CPU affinity, and clock source. Do not run this step against a mounted, allocated, or otherwise in-use DAX range.

Artifact Execution

Build and self-test the benchmark. The self-test uses anonymous memory and does not prove cross-host CXL operation:

```
cmake -S Ocean -B Ocean/build \
-DLEGOMEM_BUILD_QEMU_INTEGRATION=OFF
cmake --build Ocean/build \
--target cxl_switch_lock_bench_mpi -j2
Ocean/build/microbench/cxl_switch_lock_bench_mpi \
--self-test --iterations 32
```

Then set the exact DAX character device, hostfile, iteration count, map offset, and map size in the working configuration. Collection is fail-closed: enable DAX write acknowledgement and include the matching command-line flag:

```
acknowledge_dax_writes = true
--acknowledge-dax-write
```

The supplied example limits the writable mapping to a page-aligned 2 MiB range. After verifying that range is safe, run:

```
python3 Ocean/script/reproduce_figures_6_9.py \
all --fig 9 --config figures_6_9.toml
```

The benchmark measures sender overhead o_s , receiver overhead o_r , propagation latency L , streaming gap g , bandwidth, and contention as effective utilization increases. The analysis derives the measured and calibrated parameter tables rather than embedding the paper’s fitted values in the plotter.

Artifact Analysis (incl. Outputs)

A successful run produces these normalized tables:

```
fig9_samples.csv
fig9_params.csv
fig9_contention.csv
```

It also produces Fig. 9a–c in PDF and PNG and a checksum manifest. The expected qualitative result is that the calibrated decomposition and contention curve follow measured hardware more closely than the default OCEAN parameters. Missing MPI ranks, phases, samples, DAX authorization, or an unsafe device configuration must terminate collection without publishing a normalized table.

When fresh application or physical-CXL resources are unavailable, an existing normalized Figs. 6–9 result directory can be checked and replotted without executing any workload:

```
python3 Ocean/script/reproduce_figures_6_9.py \
validate --fig all --input <run>/normalized
python3 Ocean/script/reproduce_figures_6_9.py \
plot --fig all --input <run>/normalized \
--output <run>/replot --format pdf --format png
```

This plot-only path must retain the source labels in the normalized tables and must not be described as a fresh experimental reproduction.